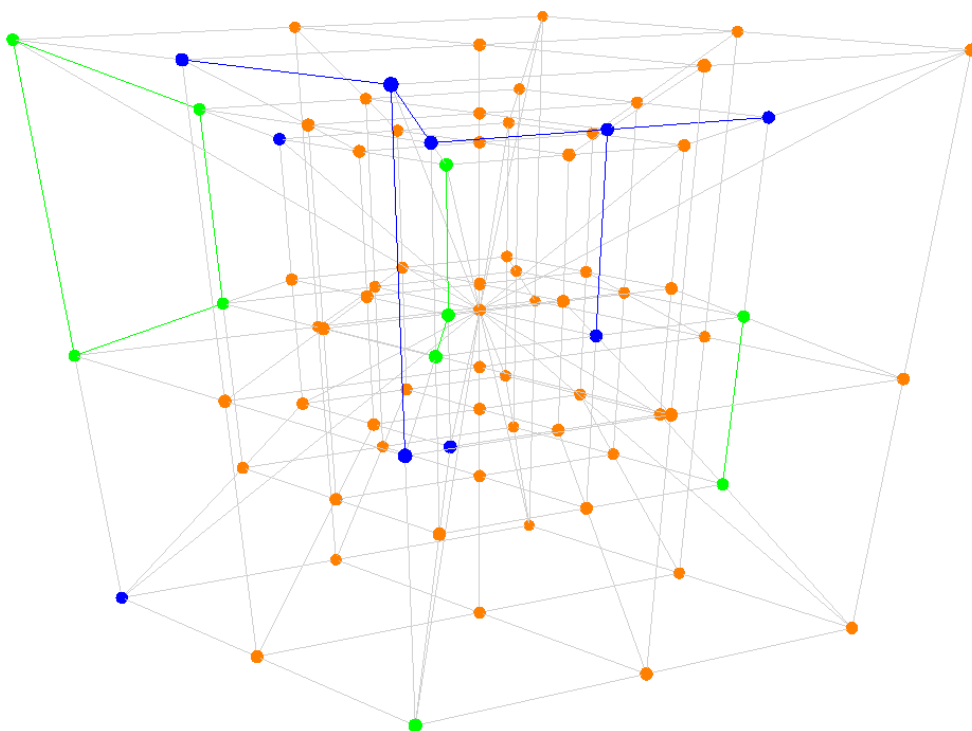




Zápočtový program

Hra 4D piškvorky v jazyce Python

Anotace

Cílem projektu je vytvořit hru schopnou graficky vizualizovat 3D projekci 4D hrací plochy modifikované hry Tic-Tac-Toe neboli piškvorek 3x3. Součástí projektu je nalezení vhodného řešení převedení této 2D hry do 4D prostoru, aby jej bylo možné rozumně vykreslovat pomocí grafické knihovny. Dále nalezení vhodné reprezentace aktuálního stavu herní plochy a detekce výhry. Hra nabízí jak možnost lokálního multiplayeru, tak hru proti počítači, která je řešená pomocí minimaxového algoritmu s omezenou hloubkou vyžadující reprezentaci možných herních stavů a heuristickou funkci k ohodnocení těchto stavů.

Motivace

Záznamy o konceptech vyšších dimenzí jsou známy již z období antiky, kdy Parmenides přešel s filosofií, ve které je realita jeden neměnný sjednocený celek a vnímání změny a pohybu jsou jen iluzí způsobené omezeností našich smyslů tuto realitu vnímat. [1, pp. 23-27]

Bližšímu matematickému zkoumání byly vyšší dimenze podrobeny v 19. století. Nejvíce toto téma proslavil Charles Howard Hinton, autor pojmu tesseract, který představil ve své knize Nová éra myšlení, ve které se snaží vybudovat intuici pro čtyřrozměrný prostor. „*The task was to obtain a sense of the properties of higher space, or space of four dimensions, in the same way as that by which we reach a sense of our ordinary three-dimensional space. I now prefer to call the task that of obtaining a familiarity with higher matter, which shall be as intuitive to the mind as that of ordinary matter has become.*“ [2, p. 13]

Cílem adaptace tohoto nového myšlení bylo vymanit se z omezení 3D prostoru a dosáhnout hlubšího porozumění reality měnícího způsobu vnímání a interakce s ní.

Mimo metody pro vybudování intuice o čtvrté dimenzi popsané v jeho knize vybuďoval Hinton prolézačku pro své děti s myšlenkou, že když se děti během svého vývoje budou pohybovat plně v 3D prostoru, který prolézačka z na sebe kolmých bambusových tyčí reprezentovala, bude pro ně následně snazší se adaptovat na čtvrtou dimenzi. K tomu měla sloužit hra pro děti, ve které se měli co nejrychleji dostat na zvolanou 3D souřadnici části prolézačky. Koncept dětských prolézaček později patentovaný jeho synem je jen jedním z příkladů pokusů o rozvinutí nástrojů pro povýšení dosavadního vnímání reality. [3]

Mezi další novodobé projekty patří 4D Miner (4D verze hry Minecraft) od vývojáře Mashpoe nebo hra 4D Minesweeper od autora Julian Schlüntz. Cílem tohoto projektu je přinést další adaptaci známé hry ve čtvrté dimenzi a napomoci tak již započaté revoluci myšlení a vnímání reality.

Použité technologie a jak hru spustit

Program byl vytvořen v jazyce Python verze 3.9.13 Využívá jeden externí balíček Ursina verze 7.0.0. Ursina (celým názvem Ursina engine) je open source projekt, jehož cílem je vytvořit herní engine schopný pohánět 2D a 3D hry doplněné o UI. Pro tento projekt byl vybrán díky jeho snadnému použití pro vykreslování jednoduché 3D reprezentace herní plochy.

Ke spuštění projektu je nutné mít nainstalovaný Python. Poté je v naklonované složce projektu z GitLabu na adrese: gitlab.mff.cuni.cz/francvoj/4d-tic-tac-toe nutné vytvořit Python virtual environmen, což lze v kořenové složce projektu provést následujícím příkazem.

```
| python3 -m venv venv
```

Dále je nutné vzniklé virtuální prostředí aktivovat

```
| source ./muj-venv/Scripts/activate
```

A nainstalovat závislosti

```
| pip install -r requirements.txt
```

Nyní už je možné projekt spustit příkazem

```
| python main.py
```

Rozšíření Tic-Tac-Toe o dvě dimenze a vizualizace hry

Klasická hra Tic-Tac-Toe se hraje v mřížce 3x3 ve dvoudimenzionální rovině. Herní plocha má podobu tabulky, do které se umísťují symboly hráčů, jde o kolečka a křížky. Začíná hráč, na kterého připadl symbol kolečka, ten ve svém tahu položí jeden svůj symbol na volné políčko a následuje tah druhého hráče. Cílem hry je na herní ploše vytvořit souvislou řadu, sloupec nebo diagonálu vyplněnou symboly daného hráče. Když dojde k zaplnění herního pole, aniž by taková souvislost vznikla, jedná se o remízu.

Jedním z možných řešení reprezentace vícedimenzionální herní plochy by bylo ji reprezentovat standardním způsobem pomocí 2D tabulky, přičemž by bylo nutné mít těchto tabulek více a každá z nich by reprezentovala jen určitý řez plochy větší dimenze. Toto řešení by bylo rozumně aplikovatelné, jednalo by se o navýšení jen o jednu dimenzi a v případě tohoto projektu by bylo nepřehledné a v rozporu s motivací obohatit intuici o fungování vyšších dimenzí, k čemuž by bylo vhodnější danou dimenzi reprezentovat s co nejmenším zkreslením.

2D reprezentace tabulkou se snaží ohraničit prostor, do kterého hráč umísťuje svůj symbol, toto řešení by už pro 3D reprezentaci nebylo optimální. Jednalo by se o 3 x 3 x 3 krychli, kde by dané prostory byly ohraničené rovinami, což by při nejmenším zakrylo prostřední krychli. Vhodnější by bylo namísto rovin zobrazovat jen obrys prostoru vymezeného pro herní symbol. Zobrazovaly by se tak jen úsečky hran krychlí vymezujících daný prostor. Informace tom, které trojice sousedních polí mohou po vyplnění být výherními, zůstávají čitelné z uspořádání herního pole. V případě projekce 4D do 3D by informace o tom, která políčka spolu sousedí (v úvahu je nutné vzít i diagonální sousedy), přehledné a snadno čitelné nezůstaly. Útvary reprezentující prostor pro umístění symbolu navíc mohou být různě deformovány napříč herní plochou a v případě tesseractu by počet hran ohraničujících takový prostor vzrostl z 12 v případě krychle na 32.

Z uvedených důvodů se jako rozumné řešení jeví zjednodušit reprezentaci herní plochy, ve které oproti vykreslování prostoru pro symboly se políčko pro jeden symbol zjednoduší na jeden vrchol. Symboly kolečka a křížku jsou dvoudimenzionální a bylo by nutné pro ně vytvořit ekvivalentní objekt ve 4D. Vzhledem ke snaze o co nejjednodušší reprezentaci herního pole je možné rozlišení, je-li pole volné nebo jednoho ze dvou hráčů, zjednodušit do 3 možných barev vrcholu reprezentujícího políčko herní plochy. Sousednost vrcholů je pak možné reprezentovat hranami, kdy hrana značí, že vrcholy, které propojuje, spolu sousedí. Tyto hrany dává smysl vytvářet jen pro ty případy sousednosti, které mohou být součástí výherní posloupnosti. Pro lepší přehlednost je možné tyto hrany obarvit barvou, spojují-li vrcholy obsazené stejným hráčem.

Tvar herní plochy

Klasické piškvorky hrané ve 2D mřížce $n \times n$ se nabízí do 3D převést vytvořením krychle $n \times n \times n$. V této reprezentaci zůstala obsažena několikrát původní mřížka, ke které přibyl další možnosti propojení do další dimenze, teoreticky je ale stále možné si jednou z těchto mřížek vybrat, nevyužít dalších možných propojení a hrát hru stejným způsobem, jako tomu bylo ve verzi o dimenzi nižší. V případě 4D by se jednalo o tesseract, který výše popsané vlastnosti také splňuje. Celkově bude herní pole tvořit $3 \times 3 \times 3$ herních polí (vrcholů).

Určit počet hran, tedy počet případů, ve kterých spolu dvě pole herní plochy sousedí tak, že mohou vytvořit výherní posloupnost, tak snadné nebude. V původní 2D hře výherní posloupnosti mohou tvořit řadu sloupec nebo diagonálu mřížky 3×3 . Pro případ řady a sloupce lze pro daný prostor namapovat tyto směry na souřadnicové osy daného prostoru. Ty by pro 2D prostor šlo reprezentovat vektory $(1, 0)$ a $(0, 1)$. Analogicky by pro 3D byl přidán další směr dopředu a dozadu a vektory by měly tvar $(1, 0, 0)$, $(0, 1, 0)$ a $(0, 0, 1)$. „On the assumption of a fourth dimension we have to suppose a fourth axis, which we will call AW. It must be supposed to be at right angles to each and every of the three axes“ [1, p. 11]

Ve čtvrté dimenzi jsou 4 možné směry. Tento prostor lze popsat pomocí čtyř souřadnicových os, které lze reprezentovat vektory o čtyřech složkách, ze kterých je právě jedna rovna jedné. Pro tesseract o straně délky 3 existuje 81 herních pozic s indexy $[t][t][t][t]$ $t = \{0, 1, 2\}$. Vezmeme-li vektor určující směr (tvořen jen 0 nebo 1) např. $(1, 0, 0, 0)$, víme, že existuje-li výherní kombinace ve směru daného vektoru, pak obsahuje prvek $[0][t][t][t]$. K němu lze 2x přičíst daný vektor a vyjde platná souřadnice jiného prvku. Takových možných počátečních pozic je $1 \cdot 3 \cdot 3 \cdot 3$ a každá z nich přispěje dvěma hranami. Pro výpočet celkového počtu hran odpovídajícímu směřům souřadnicových os prostoru se hodí pozorování, že vektory těchto směrů obsahují samé nuly a jednu jedničku, a tudíž jsou všechny jen permutacemi těchto prvků. Z toho plyne, že počet možných počátečních pozic je pro všechny permutace daného vektoru stejný. Výsledek bude počet možných permutací daného vektoru vynásobený počtem možných počátečních pozic výherní trojice dále vynásobený počtem hran vzniklých každou výherní trojicí.

$$\# \text{ hran reprezentujících nediagonální výherní trojici} = 4 \cdot 27 \cdot 2 = 216$$

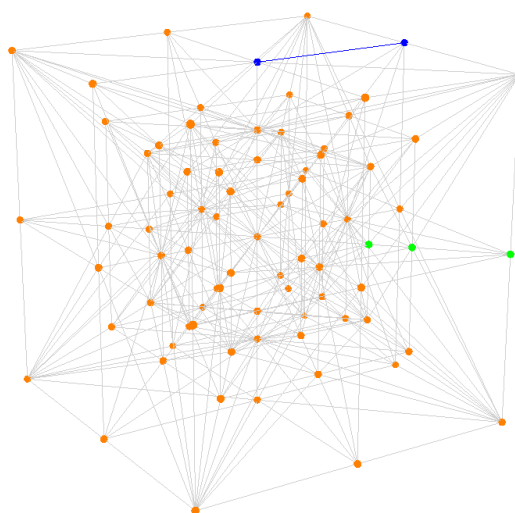
Zjistit počet hran reprezentujících diagonální výherní trojici je obtížnější. V původní 2D reprezentaci jsou diagonální směry tvořené vektory, ve kterých jsou obě složky nenulové. Jejich hodnota je rovna 1 nebo -1. Při permutování je nutné dát pozor, že na každý směr reprezentující diagonální trojici vyjdou dva permutované vektory, které se liší násobkem čísla -1. Ve 3D krychli máme 8 diagonál, kterým odpovídají třísložkové nenulové směrové vektory, nicméně stále je možné zahrnout i směrové vektory z dimenze nižší, tedy jedna složka je nulová a zbylé dvě jsou 1 nebo -1. Bylo-li by umožněno toto dědění diagonálních výherních hran, výpočet pro 4D hrací plochu by vypadal pro každý směrový vektor s 0, 1, 2 nulovými složkami následovně.

$$\# \text{možných hodnot nenulových prvků (vyděleno dvěma)} \cdot \# \text{možných rozmístění nulových prvků} \\ \cdot \# \text{počet možných výchozích pozic} \cdot 2$$

Počáteční pozice pro směrový vektor odpovídá pozici, ve které může být na pozici, ve které má vektor 0, libovolná hodnota, na pozici s 1 musí být 0 a na pozici s -1 musí být 2.

$$\left(\frac{2^4}{2} \cdot 1 \cdot 1 + \frac{2^3}{2} \cdot 4 \cdot 3 + \frac{2^2}{2} \cdot \frac{4!}{2! \cdot 2!} \cdot 3^2 \right) 2 = (8 + 48 + 108) 2 = 328$$

Celkově by takový graf obsahoval 544 hran viz obr. 1. Z obrázku je také patrné, že takové množství hran není přehledné.

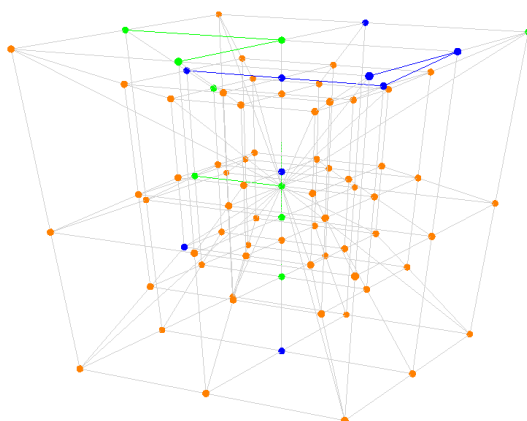


Obrázek 1 - Herní plocha se všemi diagonálními směry umožňujícími vytvoření výherní trojice

Pro zvýšení přehlednosti herní plochy je nutné snížit počet vykreslovaných hran, to by bylo možné udělat zobrazováním pouze relevantních hran, což by mohlo být implementováno tak, že se budou zobrazovat hrany vedoucí z vrcholu v okolí kurzoru myši. To by sice vedlo k lepší orientaci v zobrazených hranách, ale negativně by to ovlivnilo fungování hry, ve které by hráč musel kontrolovat vrcholy v okolí vrcholů protihráče, aby zjistil, jestli nemá někde zablokovat možnou výherní trojici, což není herní mechanika, která by podle mého názoru zlepšovala herní zážitek z této hry.

Druhá možnost je snížit počet těchto hran tvořících herní plochu a tím i nevyhnutelně snížit počet možných výherních trojic, které lze na dané herní ploše vytvořit. Z uvedeného výpočtu lze vyčíst, že z diagonálních směrů nejvíce hran tvoří ty zděděné z původní dvoudimenzionální hry, tedy ty, jejichž směrové vektory mají právě dvě složky nulové. Nejméně přispívají z diagonálních směrů hrany procházející všemi čtyřmi dimenzemi, které přispívají 16 hranami. Nabízí se tedy použít jen diagonální směry procházející čtyřmi dimenzemi, tím by ale herní plocha neobsahovala původní 3×3 herní plochu, na které by bylo možné hrát s odmyšlením přidaných spojení stejně, jako tomu bylo v původní hře, jelikož by diagonální spojení chyběla. Přes to jsem se pro tuto volbu rozhodl, jelikož všechny hrany jsou stále viditelné a herní plocha

je přijatelně přehledná. S tímto rozhodnutím hraní plocha obsahuje 232 hran a 81 vrcholů. Její podoba je vyobrazena na obrázku 2.



Obrázek 2 – Herní plocha s osmi diagonálami

Generování těchto hran je prováděno obdobným způsobem, jako byl popsán postup výpočtu jejich počtu. Hraný rovnoběžné se souřadnicovými osami čtyřdimenzionálního prostoru jsou generovány v cyklu společně s vrcholy. Diagonální hrany jsou generovány ze směrového vektoru, který je permutován a jeho případné násobky -1 již vygenerovaného vektoru jsou zahozeny. Ke každé vhodné vygenerované permutaci jsou nalezeny možné počáteční vrcholy, ke kterým je možné daný vektor dvakrát přičíst a vyjde platná pozice vrcholu herní plochy. Toto přičtení se provede a nové hrany jsou přidány.

Ovládání herní plochy

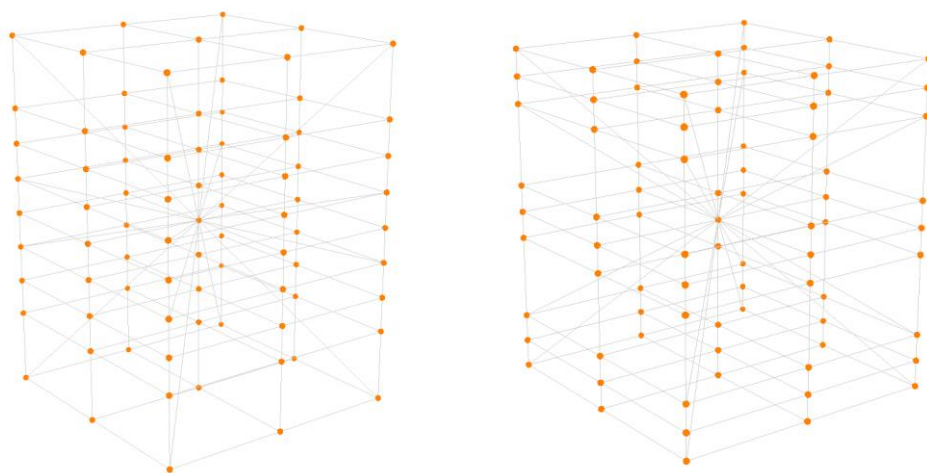
3D projekce 4D herní plochy je zobrazována na monitoru, který je schopný zobrazovat svůj obsah pouze ve 2D. Proto hra umožňuje uživateli pomocí myši 3D vykreslovaný 3D objekt otáčet ve dvou osách. Provádění těchto rotací má na starosti objekt OrbitController, který se nachází ve stejnojmenném souboru. Ten generuje matici, kterou se pak počáteční pozice každého prvku vynásobí. Při testování se metoda, kdy byla vygenerována matice zohledňující pouze relativní rozdíl od předchozí pozice, neosvědčila, jelikož po provedení určitého počtu rotací se nejspíše nepřesnosti způsobené reprezentací desetinných čísel z každé rotace nasčítaly a herní plocha se začala nepatrně otáčet i kolem třetí osy, kolem které model rotovat neměl.

K výpočtu rotace má tento objekt přístup k pozici kurzoru myši, ze kterého vypočítá vektor posunu myši od předchozí uložené pozice. Aby rotace působila plynule, je rotace spočítána jako lineární interpolace od předchozí rotace k požadované rotaci, kdy parametr t je součinem uplynulého času od předchozí aktualizace rotace s konstantou stanovující rychlost rotace. Díky tomu rotace pokračuje i když už byl pohyb myši dokončen a plynule zpomaluje.

Mimo zmiňované rotace ve dvou 3D osách je možné měnit projekci 4D objektu do 3D prováděním jedné rotace, která mění souřadnice y, z, w a tudíž ovlivňuje pozici prvků v ose w, což předchozí dvě rotace nedělaly. Tyto rotační matice a maticové násobení provádí funkce souboru matrix.py. Vzniklé 3 matice rotace velikosti 4x4 jsou vynásobeny a funkce getOrbitMatrix objektu OrbitController vrací tuto matici rotace, kterou získává objekt Game, který obsahuje metodu update. Tato metoda je dokola volána knihovnou Ursina. Při každém volání si tuto matici vyžádá a poté jí vynásobí původní 4D pozici každého objektu herní plochy.

Projekce z 4D do 3D

Původní 4D pozice objektů reprezentována jako vektor 4×1 je po vynásobení maticí rotace velikosti 4×4 zleva je stále reprezentována čtyřdimenzionálním vektorem. nicméně Ursina engine pracuje pouze s 3D vektory udávajícími pozici. Proto je potřeba vypočtený 4D vektor nové pozice převést na vektor udávající pozici ve 3D prostoru. Jako nejjednodušší řešení se nabízí nepoužít poslední souřadnici. Pak herní plocha vypadá jako 3 krychle tvořena 9 menšími krychlemi, které se při zvolené 4D rotaci navzájem pohybují podél osy y (viz obr. 3).

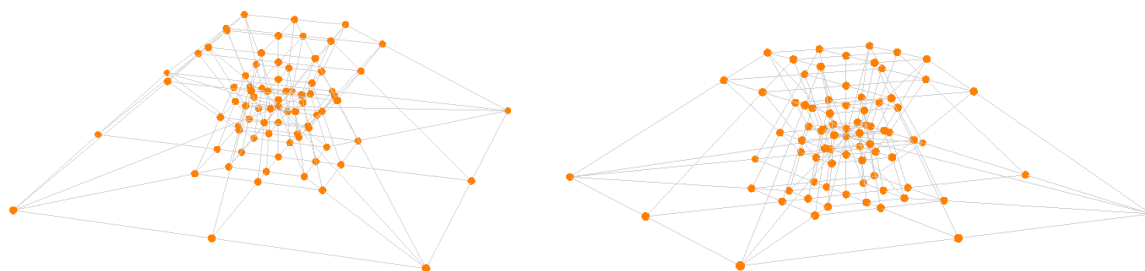


Obrázek 3 – Vizualizace herní plochy vypuštěním čtvrté souřadnice

Chtěli bychom lepší převedení 4D pozice do 3D pozice, ve kterém se nějakým způsobem projeví vliv všech čtyř složek vektoru pozice ve 4D. K tomu lze použít stereografickou projekci, která má podobu následující matice, kterou je 4D vektor zleva vynásoben.

$$\begin{pmatrix} \frac{1}{l_w - w} & 0 & 0 & 0 \\ 0 & \frac{1}{l_w - w} & 0 & 0 \\ 0 & 0 & \frac{1}{l_w - w} & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

W je hodnota pozice vektoru ve čtvrté ose a l_w značí vzdálenost světla podél osy W . Pozice světla ovlivňuje stín, a jehož o dimenzi chudší pozice je reprezentována ve výsledku maticového součinu. [4] Výsledná projekce je vyobrazena na obr. 4.

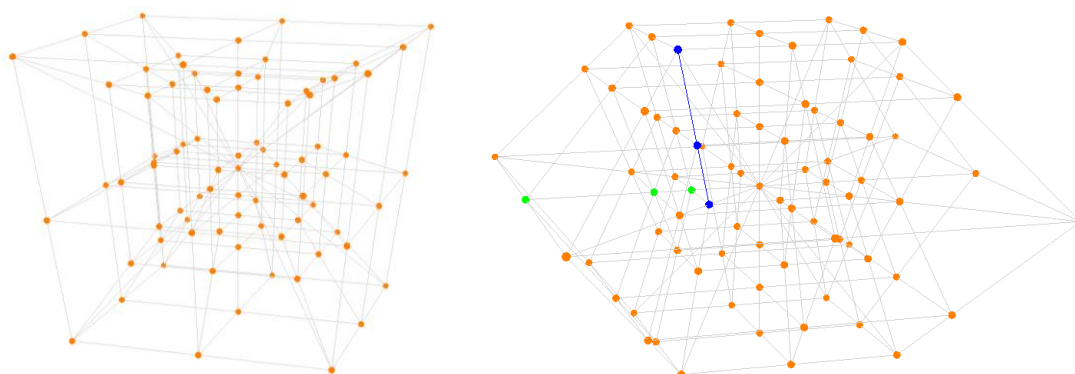


Obrázek 4 – Stereografická projekce herní plochy

Z obrázků je patrné, že hlavní koncentrace vrcholů je v oblasti středu a zejména osm nejvzdálenějších vrcholů mají výrazně delší rozestupy než kterékoliv jiné vrcholy. Měla-li být celá herní plocha viditelná, vrcholy ve středové oblasti by byly velmi blízko sebe, což by nepůsobilo přehledně, a oblast mezi středem a nejvzdálenějšími vrcholy by byla prázdná. Vhodné by bylo projekci upravit tak, aby byly vrcholy rozprostřeny rovnoměrněji.

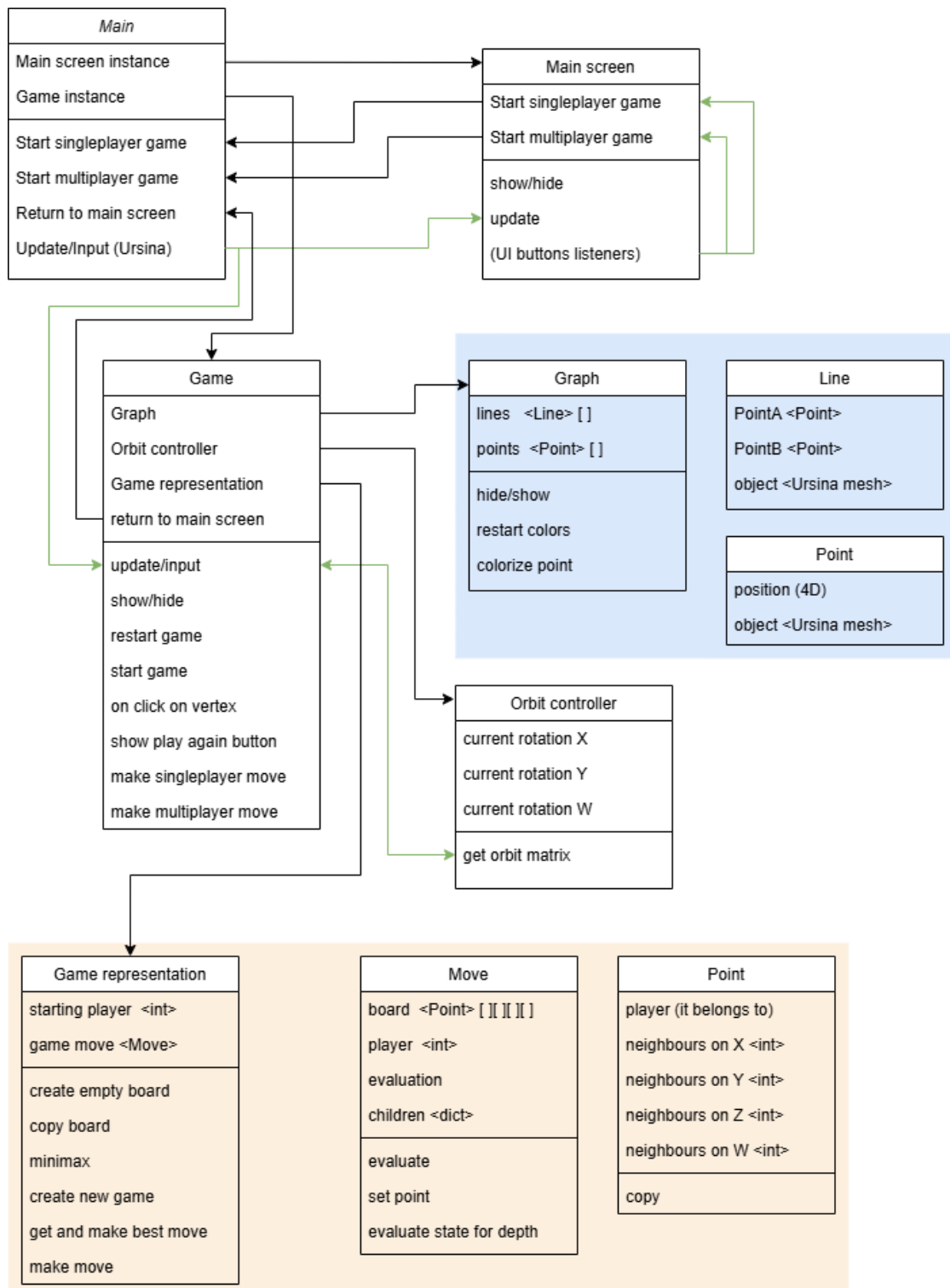
Lze vypočítat, že míru vybočení udává pozice w . Čím je tato hodnota pro daný vrchol větší, tím více vybočují krajní vrcholy a tím více se koncentrují vrcholy uprostřed. Bude-li nastavena tato hodnota na 0, bude chování projekce stejné, jako v případě vypuštění poslední souřadnice (pouze l_w bude měnit měřítko). Proto se nabízí vzdálenost hodnoty w od 0 uměle snižovat, což lze udělat vydělením vhodnou konstantou. Takto lze dosáhnout rovnoměrnějšího rozmístění vrcholů po herní ploše (viz obr. 5). Matice upravené stereografické projekce má následující podobu.

$$\begin{pmatrix} \frac{1}{l_w - \frac{w}{k}} & 0 & 0 & 0 \\ 0 & \frac{1}{l_w - \frac{w}{k}} & 0 & 0 \\ 0 & 0 & \frac{1}{l_w - \frac{w}{k}} & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$



Obrázek 5 – Upravená stereografická projekce herní plochy

Struktura projektu



Soubor main spouští celou hru, ovládá, jestli se zobrazuje hlavní menu, a obsahuje funkce pro zahájení hry. To provádí voláním metod instancí tříd Game a Main screen. Také obsahuje funkci update, která je volána balíčkem Ursina. V této funkci jsou volány update metody instancí objektů. Třída Game uchovává dvě reprezentace hry, jednu pomocí instance třídy Graph, která obsahuje instance 3D zobrazovaných objektů, tedy vrcholů a hran. Vrcholy jsou uloženy v 4D poli podle jejich pozice a hrany v seznamu. Objekt reprezentující hranu obsahuje souřadnice vrcholů, které spojuje, a zobrazovaný objekt hrany. Třída Graph také tuto vizuální reprezentaci generuje ve svém konstruktoru na základě argumentů jako je vzdálenost mezi vrcholy a jejich poloměr. Druhá reprezentace herní plochy je více symbolická, zabývá se pouze vrcholy a slouží k detekci konce hry nebo k reprezentaci možného budoucího stavu hry pro minimax algoritmus.

Reprezentace stavu hry

Mimo zmíněnou třídu Graph, starající se o instance 3D objektů balíčku Ursina, jsou stavy hry ukládány jako instance třídy Game representation, ve které jsou uchovávány informace o hráči, který v daném stavu provádí tah, a stavu herní desky, která je reprezentována v 4D poli. Každé políčko v tomto poli je instancí třídy Point, která obsahuje informace, jestli dané políčko nepatří jednomu z hráčů, a kolik je políček patřících stejnému hráči spolu souvisle sousedících v daných osách. Šlo původně o snahu zefektivnit detekci výhry před odstraněním mnoha možných diagonálních směrů, jelikož by v takovém případě bylo nutné těmito všemi možnými směry, ve kterých může být výherní trojice, procházet celou herní plochu ze všech možných startovacích pozic. Tato alternativa nabízela projít všech 81 vrcholů a u každého zkontrolovat, jestli se některá z jeho hodnot pro některý směr nerovná třem.

Výpočet nutných směrů pro 4D prostor, ve kterém bychom umožňovali všechny diagonální směry by bylo možné provést pozorováním, že pro danou dimenzi vytvoříme mřížku o velikosti hrany 3. Pro 2D by to byla mřížka 3x3 a pro 3D by se jednalo o 3x3x3. V každé takové reprezentaci máme prostřední pozici, ze které všechny možné diagonální směry vedou do některé ze zbývajících pozic. Jelikož by bylo zbytečné odlišovat směry, které jsou navzájem opačné, a pro každý z těchto směrů lze najít směr opačný, lze počet takových směrů spočítat pro dimenzi n takto:

$$\#směrů = \frac{3^n - 1}{2}$$

Pro čtyřdimenzionální prostor vychází 40 unikátních směrů. K detekci výhry by tak bez optimalizace bylo potřeba $81 \cdot 40$ kontrol, což vychází hůře než procházení celé herní plochy z možných startovních pozic pro všechny směry, jelikož např. pro směr (1, 1, 1, 1) existuje jen jeden možný počáteční bod, ale v předchozí reprezentaci budou tento směr obsahovat i všechny ostatní vrcholy. To by šlo řešit tím, že by každý vrchol obsahoval jen směry pro něj z hlediska možné výhry relevantní. Tak by se efektivita obou přístupů z hlediska času dorovnala. Nicméně velká výhoda pamatování si délek souvislých úseků patřících stejnému

hráči pro daný směr spočívá v možnosti zjistit výherní stav rychleji v případě, že víme, že hra v předchozím tahu neskončila, a také známe-li pozici právě zahraného vrcholu. Pak stačí zjistit, jestli daný vrchol nesousedí v nějakém směru s vrcholem, pro který délka řady pro daný směr vychází 2. Pokud daným tahem hra nekončí, je nutné projít všechny sousedy a případně aktualizovat jejich záznam o délce souvislého úseku v daném směru, kterého jsou součástí. Pak je ale nutné tuto hodnotu opravit u každého vrcholu, který je součástí tohoto úseku.

Minimaxový algoritmus pro hledání vhodného tahu

Třída Game representation obsahuje metody pro nalezení vhodného tahu pomocí minimax algoritmu. K tomu slouží především rekurzivní metoda minimax, která dostává jako argumenty stav herní plochy, zbývající hloubka pro omezení hloubky rekurze, indikátor, jedná-li se o první vrstvu, z jejíž možných stavů vybíráme ty nejvhodnější. Dále obsahuje argumenty pro alpha a beta prořezávání.

Jelikož i tato mutace hry piškvorky je konečná (někdo vyhraje nebo se zaplní herní plocha), jde o hru s úplnou informací na rozdíl od některých karetních her a jde o hru s konečným součtem, je možné k nalezení nejlepšího možného tahu v dané pozici použít algoritmus minimax. Ten v prvním tahu nové hry udělá 81 větvení a v každém dalším tahu mu zbývá o jedno políčko k vyzkoušení méně. Maximálně by tak spodní vrstva měla 81! stavů, což není příliš přesné, jelikož by to znamenalo pokračovat ve hře i po výhře některého z hráčů. I tak je možných větvení mnoho. Pro omezení hloubky se osvědčila hodnota 3, na kterou je potřeba čekat ve většině případech pod jednu sekundu. V případě, že na herní ploše je již 7 polí zabráno, takový algoritmus musí vyzkoušet maximálně $74 + 74 \cdot 73 + 74 \cdot 73 \cdot 72 = 394\,420$ stavů, což není tak velké číslo vzhledem k řádu operací, které zvládne provést procesor za sekundu. Pro hloubku 4 toto číslo vychází 28 009 444, což už není vypočitatelné v době, se kterou by bylo pohodlné proti počítači hrát. Pro zefektivnění by se nabízelo snížit počet zvažovaných pozic, což je vzhledem k malým rozměrům hrací plochy ošemetné. Svou roli v neefektivitě také hraje fakt, že se objekt reprezentující hrací plochu při každém vygenerování nového tahu klonuje.

Když algoritmus dorazí na poslední zkoumanou vrstvu, je tato vrstva ohodnocena heuristickým algoritmem. Ten je implementován jako metoda třídy Game state, která dostane jako argument herní plochu a id hráče, pro kterého hodnocení počítá. Samotný výpočet projde všechny možné výherní trojice pozic a přidá ohodnocení váhy 1, pokud v dané trojici má daný hráč jedno své pole a zbylá pole jsou volná. Ohodnocení 4 mají trojice pozic, ve kterých daný hráč zabírá 2 pole a zbylé jedno pole je volné, nicméně v těchto případech je odečítáno ohodnocení 6 od skóre protihráče, aby jeho priorita byla těmto případům zabránit s výjimkou situací vlastní výhry.

Bohužel k vyhodnocování těchto hodnot se nehodí struktura, ve které si jednotlivé pozice pamatují délky souvislých úseků stejné barvy v daném směru, jelikož není vyžadována souvislost a záleží, jestli je pozice neobsažená v tomto úseku zabraná protihráčem.

Závěr

Výsledkem je fungující hra s grafickým UI, která nabízí možnost lokálního multiplayeru a hře proti počítači. Nepodařilo se zjistit, jestli v této upravené verzi piškvorek má výhru garantovanou první hráč, ale při hře proti počítači je možné vyhrát a jelikož se počítač ze svých chyb neučí, je možné výherní postup libovolněkrát opakovat. Počet tahů dopředu, které počítač analyzuje, není příliš velký a nabízí se vhodnou další optimalizací společně využitím heuristických algoritmů tuto hloubku navýšit.

Zároveň hra i přes všechny snahy o přehlednost umožňuje překrývání hran a vrcholů, kdy při kliknutí na shluk překrývajících se vrcholů dojde k obarvení jednoho z nich a není podmínkou, aby byl tento vrchol vidět. Jinak zbytek hry působí z grafického hlediska vydařeně s některými propracovanými detaily, jako je plynulá animace rotací. I přes zmíněné nedostatky projekt naplnil motivaci vedoucí k jeho vzniku a testovacími hráči byl kladně hodnocen.

Bibliografie

- [1] C. H. Hinton, *The fourth dimension*, Swan Sonnenschein & Co. Limited, 1906.
- [2] C. H. Hinton, *A New Era of Thought*, Londýn: Swan Sonnenschein & Co. Ltd., 1888.
- [3] „J” is for Jungle Gym,” *The Gazette*, Fall 1997.
- [4] L. Labs, „Understanding 4D -- The Tesseract,” 19 11 2016. [Online]. Available: <https://www.youtube.com/watch?v=iGO12Z5Lw8s>. [Přístup získán 29 1 2025].